

Materialized Views



Learn2
CloudData

Powered by

www.learn2clouddata.com

Sujith Nair

Cloud Data Architect
Snowflake Snowpro Certified

Section Introduction–Materialized Views

- What are Materialized views ?
- Understanding limitations of Materialized views.
- LAB Materialized View
 - Create a Materialized view.
 - Test limitations of Materialized views.
 - Create clustered Materialized views.
 - Suspend and resume a Materialized view.
 - Understand auto suspension of Materialized view due to changes in the base table.
 - Analyze credit consumption by Materialized views.
- Summary.

What are Materialized views?

Materialized views can be thought of as a hybrid between a table and view. Just like a view it has a query associated with it and like a table it stores data physically in it.

When we create Materialized view on a table the Materialized view is populated by the select statement provided in the create Materialized view.

```
CREATE MATERIALIZED VIEW MVW_EMPLOYEES  
AS  
SELECT * FROM EMPLOYEES;
```

We can select data from the Materialized view with a SELECT statement

```
SELECT * FROM MVW_EMPLOYEES;
```

If the base table changes then the changes are propagated to the Materialized view automatically without any input needed from the data engineer.

Do not to use a Materialized view if the base table changes frequently.

Why do we need Materialized view ?

What are Materialized views?

Primary role of Materialized view is to help with query performance.

Area for performance improvement

- External table select performance.
- Select performance on large tables(>1 TB) by creating clustered Materialized views.
- Complex query which changes very infrequently.

Areas of Common use with Materialized views.

- Aggregates that take a long time to calculate.
- Analysis of semi-structured data.
- Query results contain a small number of rows and(or) columns relative to the base table.
- External tables and clustering large snowflake tables.

Materialized views do not support DML operations like INSERT,UPDATE and DELETE.

We can create secure Materialized views which can be shared.

Materialized views guarantee that the data selected is the latest data from the base table even if the Materialized views has not been fully refreshed . It does this by getting data from the base tables directly.

Limitations of Materialized views?

- A Materialized view can query only a single table. Joins, including self-joins, are not supported.
- A Materialized view cannot have
 - Materialized view.
 - Non-Materialized view.
 - UDTF (user-defined table function).
 - UDFs (including external functions).
 - Window functions.
 - ORDER BY clause.
 - LIMIT clause.
 - GROUP BY columns that are not within the SELECT list.
 - HAVING clauses.
 - GROUP BY GROUPING SETS.
 - GROUP BY ROLLUP.
 - GROUP BY CUBE.
 - Subqueries within a Materialized view.
 - The MINUS, EXCEPT, or INTERSECT set operators.(UNION and UNION ALL are supported)
- Many aggregate functions are not allowed in a materialized view definition.
- SUM,COUNT,MAX,MIN,AVG,COUNT_IF,HILL,STDDEV,STDDEV_SAMP,BITAND_AGG,BITOR_AGG,BITXOR_AGG,STDDEV_POP,VARIANCE,VARIANCE_POP are supported.

Summary

- Materialized view is a hybrid between a table and view, it stores data physically in it like a table and has a query associated with it which it uses to populate the Materialized view and keep the data updated.
- Materialized views are primarily used for performance improvement in External tables, Clustering large Snowflake tables to provide alternate cluster keys for select , for aggregation on very large tables that change infrequently and take a lot of time to calculate and for creation of views on semi-structured data.
- Materialized views are automatically maintained by Snowflake by auto refreshing the data.
- Data accessed from the Materialized views is always current irrespective of if the Materialized view is fully refreshed or not. This is done by accessing the base table and getting data from it in case the view is not caught up.
- Materialized views do not support joins including self joins.
- Materialized views support **GROUP BY** as long as the fields are in the select statement.
- Materialized views do not support **HAVING** clause.
- Materialized views do not support set operators like **MINUS**, **EXCEPT** or **INTERSECT**.
- Materialized views do not support **ORDER BY** or **LIMIT**.
- Materialized views do not support UDF, UDTF and Windows functions.
- Materialized views cannot include a Materialized views or regular view in its query definition.
- Materialized views support several aggregate function including **SUM** , **COUNT**, **MAX**, **MIN**, **COUNT_IF**, **AVG**, however most others are not supported.

Summary

- Functions used in a Materialized view must be deterministic. For example, using `CURRENT_TIME` or `CURRENT_TIMESTAMP` is not permitted.
- Auto refresh of Materialized views consumes credits.
- We can check the `BEHIND_BY` columns in the output to see how far behind the Materialized view is from the base table.
- We can check the `REFRESHED_ON` to see when the materialized view was last refreshed.
- We can suspend a Materialized view by using the `ALTER MATERIALIZED VIEW <name> SUSPEND`.
- We can resume a Materialized view by using the `ALTER MATERIALIZED VIEW <name> RESUME`.
- The `INFORMATION_SCHEMA.TABLES` shows Materialized views and we can use `SHOW VIEWS` to see the views.
- You cannot apply DML operation on a Materialized view including `COPY,DELETE,INSERT,MERGE` or `UPDATE` or `TRUNCATE`.
- Materialized view do not support time travel.
- If new columns are added to the base tables the new columns do not show up on the materialized view even if the the view definition has `SELECT * FROM <TABLE>`.
- If a base table is altered so that existing columns are changed or dropped, then all materialized views on that base table are suspended. This is true even if the modified or dropped column was not part of the materialized view.

Summary

- If a base table is dropped, the materialized view is suspended but not automatically dropped. The materialized view must be dropped manually or the dropped table re-created so that we do not have an invalid materialized view.
- Costs and refresh history can be tracked by looking at [MATERIALIZED_VIEW_REFRESH_HISTORY](#).
- Streams are not supported on materialized views.
- We can use Snowflake sharing to share a secure Materialized view.
- Materialized view cannot be cloned because it is a view object.

Thank you



Learn2
CloudData

Powered by

www.learn2clouddata.com

Find Us

www.learn2clouddata.com